

Contextual Bandits

And Bayesian Optimization

Claire Vernade, Alexander von Rohr

UTN

Agenda

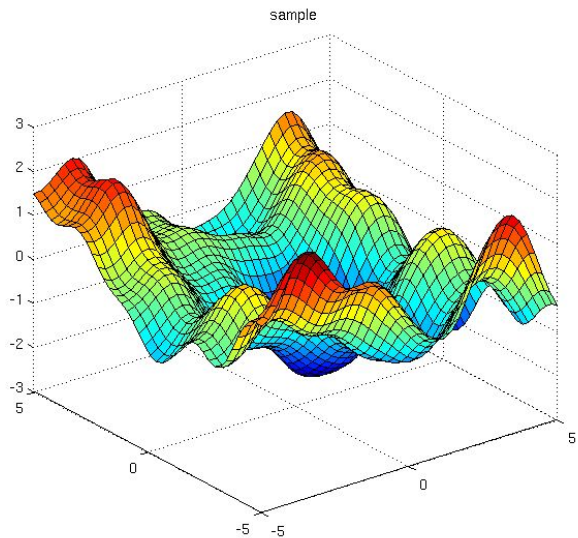
1. General introduction to bandits and contextual bandits (iPad ~45 min)
 - a. Notion of regret and best-arm identification
 - b. Exploration-exploitation (in K-armed bandits and contextual bandits)
 - c. UCB vs Thompson Sampling for K-armed bandits
 - d. Linear models, Bayesian Linear Regression and LinTS
2. From GP-UCB to BayesOpt (~45min)
 - a. Introduction to the Bayesian Optimization problem: parallels and common differences with contextual bandits (e.g. observation noise)
 - b. Expected Improvement
 - c. GP-UCB (Srinivas et al. 2009): regret minimization for BayesOpt
 - d. Beyond GP-UCB: Local search methods (GIBO)
3. Practical session: Implementation and comparison of GP-UCB and GIBO (~45min)
4. Discussion and questions: “We still don’t understand Bayesian Optimization”
– AISTATS 2026 (~15 min)

Sources and Resources

- *Bayesian Optimization* by R. Garnett. <https://bayesoptbook.com/>
 - Great illustrations and intuitions, Practical, no connections to bandits / theory
- *Gaussian Process Optimization in the Bandit Setting: No Regret and Experimental Design*. Srinivas et al. 2010. ICML
<https://arxiv.org/pdf/0912.3995>
- *The Kernel cookbook*. David Duvenaud.
<https://www.cs.toronto.edu/~duvenaud/cookbook/>
- *The Vezier Gaussian Process bandit Algorithm*. Google report.
<https://arxiv.org/abs/2408.11527>
- *We still don't understand high-dimensional Bayesian Optimization*. Doumont et al. 2026. AISTATS. <https://arxiv.org/abs/2512.00170>

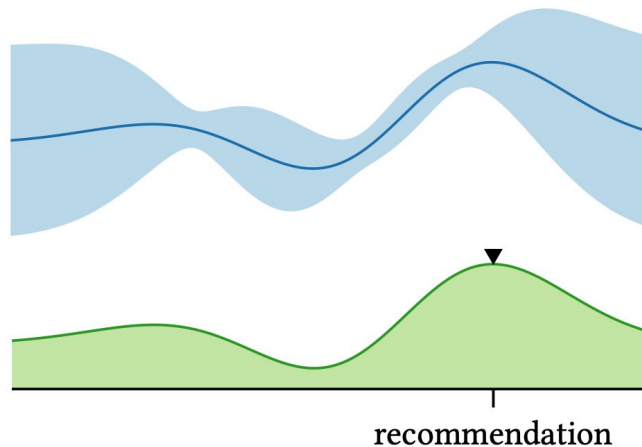
Bayesian Optimization

- Zero-th order (non-convex) optimization problem:
 - Hidden function typically non-convex
 - Can only be accessed through pointwise probing
 - No gradients!
 - Examples: hyperparameter optimization in ML, Industrial experimental design (chemistry, Robotics, etc.)
- Typically fixed-budget
- Can be cost-sensitive
- Can we do better than random search?



Bayesian Optimization as a Bandit Problem

- Hidden (smooth) function (blue)
- Learning setting: At each time t ,
 - Probe the function at point x_t (arm)
 - Observe a (noisy) value $y_t = f(x_t) + \text{noise}$
 - Update belief
- **Objective:** After T rounds (budget), return a recommendation $\hat{x}_T^*$ that minimizes
$$|\max_x f(x) - f(\hat{x}_T^*)| \quad (\text{simple regret})$$



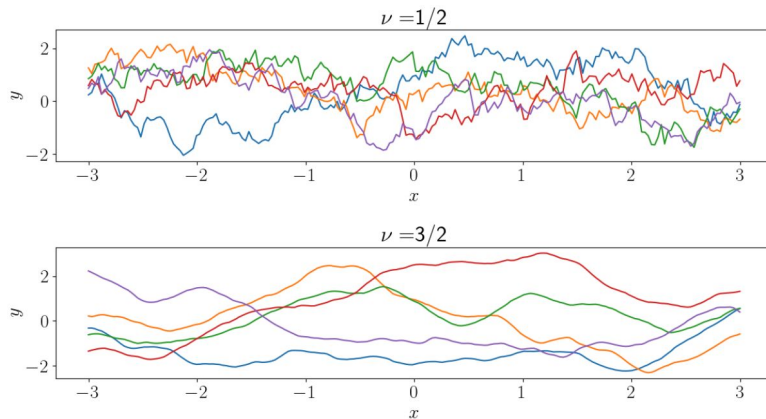
Or minimize overall

$$R(T) = \sum_{t=1}^T \max_x f(x) - f(x_t) \quad (\text{cumulative regret})$$

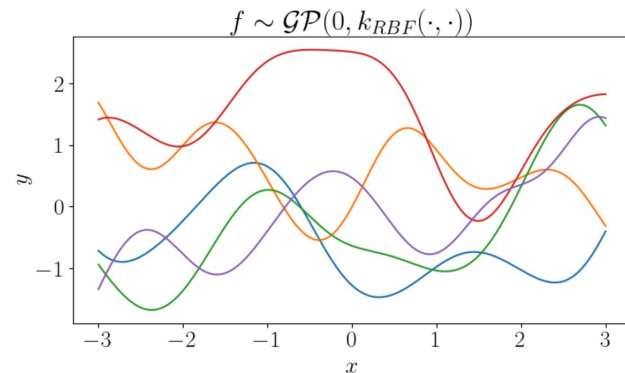
Bayesian Optimization as a Bandit Problem

- Infinitely many arms!
- ... But they locally have similar values (smoothness)
- Example: f is linear $\rightarrow$ Stochastic Linear Bandits :) $f(x_t) = x_t^\top \theta$ for $\theta \in \mathbb{R}^d$
- But we said “smooth”: encoded by kernels (prior, see tomorrow’s lecture)

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} \|\mathbf{x} - \mathbf{x}'\|}{\ell} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu} \|\mathbf{x} - \mathbf{x}'\|}{\ell} \right)$$



$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp \left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2} \right)$$



Bayesian Optimization as a Kernelized Bandit Problem

- Given a kernel / smoothness assumption : $k(x, x')$ encodes similarity
- Assume we have sampled and observed $x_1, \dots, x_t$
- The (reward) function f can be estimated at any new point x :
 - Define the kernel matrix $K_t = (k(x_i, x_j))_{1 \leq (i, j) \leq t}$ (all the “distances”)
 - For a new point x , define the similarity vector $k_t(x) = (k(x, x_1), \dots, k(x, x_t)) \in \mathbb{R}^t$
 - The value at x is a weighted average of the values at $x_1, \dots, x_t$:

$$f(x) = \sum_{i=1}^t w_t(x) y_i \quad \text{where} \quad w_t(x) = (K_t + \sigma^2 I_t)^{-1} k_t(x)$$

- Parametric vs non-parametric: parallel with Linear vs Lipschitz bandits

Gaussian Processes

- For the remainder of the lecture, fix the squared-exponential kernel:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp \left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2} \right)$$

- Probabilistic view: this defines a “Gaussian Process”
 - Predictive distribution is Gaussian: $f(x) \mid \mathcal{D}_t, x \sim \mathcal{N}(\mu_t(x), \sigma_t^2(x))$

$$\mu_t(x) = k_t(x)^\top (K_t + \sigma^2 I_t)^{-1} y_t$$

$$\sigma_t^2(x) = k(x, x) - k_t(x)^\top (K_t + \sigma^2 I_t)^{-1} k_t(x)$$

Algorithmic ingredients

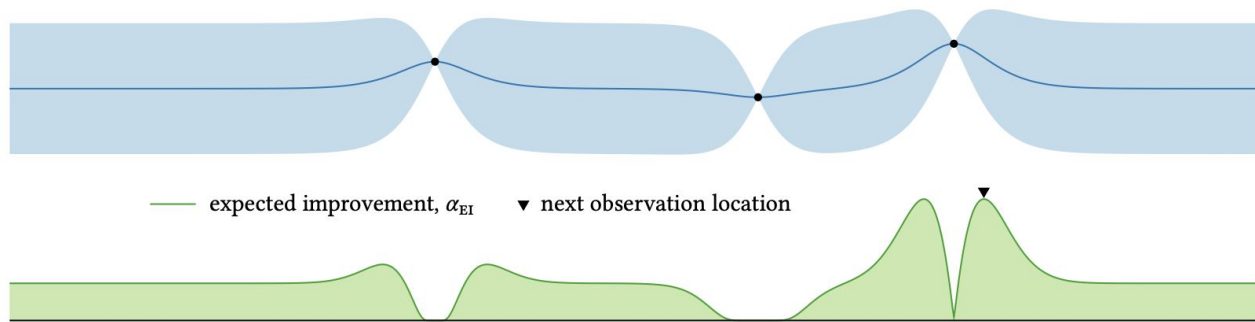
1. **Sampling rule:** How should I sample the next point x_{t+1} ?
 - a. “Acquisition function”: A “guiding” function $g(x)$ such that $x_{t+1} = \operatorname{argmax} g(x)$
 - b. “lookahead” idea: what benefit would the next point x' bring?
 - c. The acquisition function measures this benefit
2. **Decision rule** (for Best-Arm identification setting)
 - a. At the end of the sampling phase ($t=T$): choose the final answer $\hat{x}_T^*$
 - b. Typically best performing sample in the collected dataset.
 - c. Could average the iterates around max (?)
3. **Regret minimization**
 - a. Plus often a decision rule: return the highest performing point in the data set

Expected Improvement

- Denote the data collected so far $D_t = \{x_1, \dots, x_t\}$
- Define the current max: $f^+ = \max_{x \in D_t} f(x)$
- The **Expected Improvement** defines an acquisition function:

$$EI(x) = \mathbb{E}[\max f(x) - f^+] = \int_{-\infty}^{\infty} (\max y - f^+) p(y|x, D_t)$$

- Choose $x_{t+1} \in \arg \max_x EI(x)$

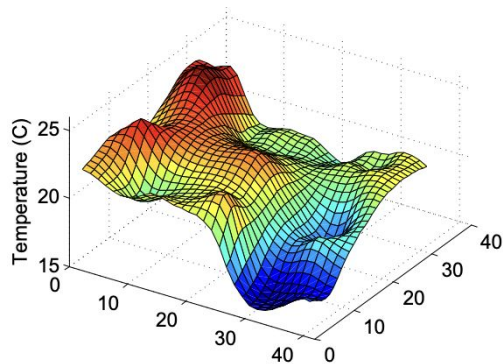


GP-UCB: Information Gain and Optimism

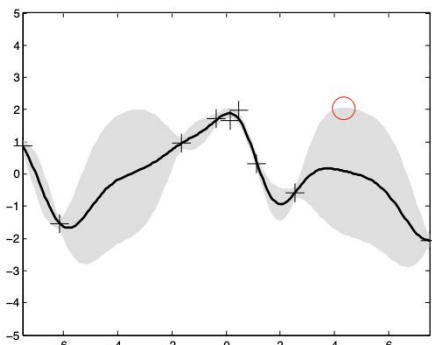
- At time t , we can compute a high-probability upper bound for each x :

$$g_t(x) = \mu_{t-1}(x) + \sqrt{\beta_t \sigma_{t-1}(x)}$$

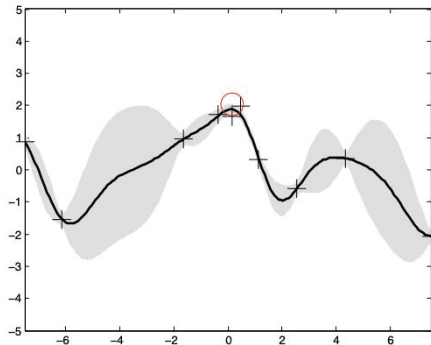
- GP-UCB: Use g as acquisition function: choose $x_t \in \arg \max_x g_t(x)$



(a) *Temperature data*



(b) *Iteration t*



(c) *Iteration $t+1$*

Figure 2. (a) Example of temperature data collected by a network of 46 sensors at Intel Research Berkeley. (b,c) Two iterations of the GP-UCB algorithm. It samples points that are either uncertain (b) or have high posterior mean (c).

Gaussian Process Thompson Sampling

- The GP defines a posterior distribution over functions:

$$f|D_t \sim \mathcal{GP}(\mu_t, \sigma_t^2)$$

- GP-TS proceeds as follows:

- Sample a function $\tilde{f}_t \sim \mathcal{GP}(\mu_t, \sigma_t^2)$
- Use $\tilde{f}_t$ as an acquisition function and optimize it: $x_{t+1} \in \arg \max \tilde{f}_t$
- Observe the **true f**: $y_{t+1} = f(x_{t+1}) + \eta_{t+1}$ and update GP posterior

Wait, what?

- Sample a function from the GP? How do you do that?
 - Discretize: choose m candidate points on a grid and sample from local posterior predictive
 - Use another layer of approximation (i.e. Random Fourier Features)

Regret guarantees for GP-UCB

With an appropriate choice of the **exploration parameter** β_T , GP-UCB satisfies, with high probability,

$$R_T = \tilde{\mathcal{O}} \left(\sqrt{T \beta_T \gamma_T} \right),$$

Where $\gamma_T = \max_{A: |A|=T} I(y_A; f)$, is the **maximal information gain**:

For a given set of acquisition points A , the mutual information $I(y_A; f)$ quantifies how much we can learn about f by observing y_A .

Intuitively, it is the “**maximal variance reduction you can get with a finite set of T samples**”.

Can we compute it?

RBF Kernel and GP-TS

For the RBF kernel in dimension d , $\gamma_T = \mathcal{O}((\log T)^{d+1})$.

Thus, the regret of GP-UCB is

$$R_T = \tilde{\mathcal{O}}\left(\sqrt{T}(\log T)^{\frac{d+2}{2}}\right)$$

Recent improvement of GP-UCB (Chowdhury & Gopalan, 2017)

$$R(T) = \tilde{O}(\sqrt{T\gamma_T})$$

They also show a (worse) upper bound for GP-TS:

$$R(T) = O(\gamma_T \sqrt{dT})$$

Optimizing the acquisition functions

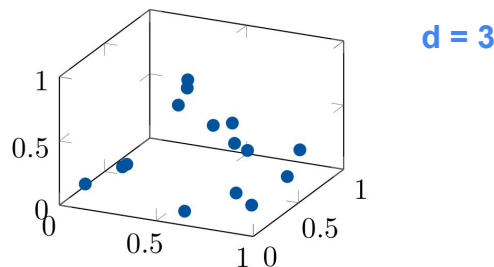
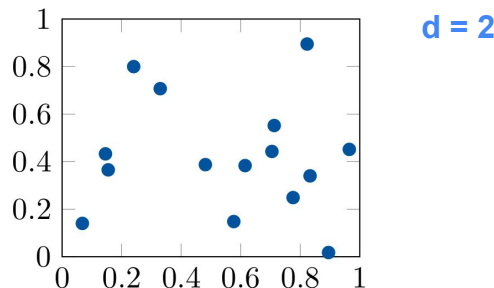
- Recall the objective was: find $x^* \in \arg \max_x f(x)$
- We have transformed it into finding, at each time t , $x_{t+1} \in \arg \max_x g_t(x)$ for some acquisition function g_t
- **What have we gained?**
- f is very costly to sample, g_t is not! And we can compute gradients :)
- Can use
 - **global non-convex** algos (“dividing rectangles” or covariance matrix adaptation evolution strategy), or
 - **local optimization** with multi-start strategies
 - See *On local optimizers of acquisition functions for Bayesian Optimization*. Kim and Choi (2020) <https://arxiv.org/abs/1901.08350>

Is this the end of the story?

- “f is very costly to sample, g_t is not!” -> REALLY??
- What if the space is **high dimensional?** ($d > 20$)
 - Global and local optimization struggle
 - Each step requires a lot of compute (to optimize g_t)
 - Then, is it still worth it compared to probing f? (“economics of BO”)
- Folklore “knowledge”: **Bayesian optimization does not work in high dimension**
- But indeed, there is more to the story...

Why go local?

- Global BO searches the **whole** domain.
- Its volume grows exponentially with the dimension. The same budget covers an interval, not a cube.
- Global acquisitions respond by exploring **more**.
- But we might not need the global optimum: a good solution near a sensible start is enough.



Two ways to localize the search

Restrict the domain: trust regions

- Keep a global-style acquisition, but **optimise it only inside a box around the current best found value**.
- Shrink the box after failures, grow it after successes.
- TuRBO — Eriksson et al., NeurIPS 2019.

Follow a descent path: local models

- Model the local structure — the gradient, or the descent sequence it induces.
- Move along it; query only to sharpen that local belief.
- **GIBO (2021)** · Most Probable Descent (2022) · MinUCB (2024) · Local Entropy Search (2025).

GIBO: Estimate the gradient from zeroth-order observations, then take a gradient step.

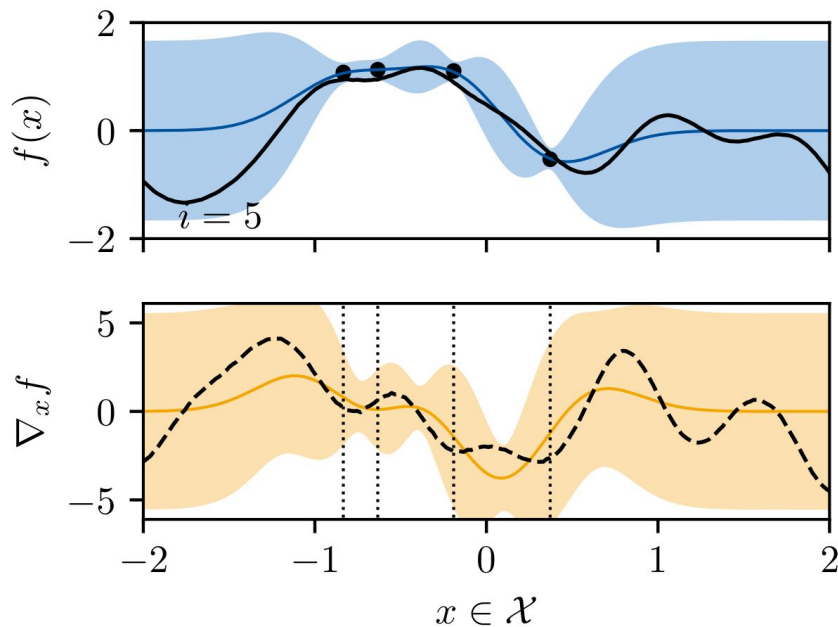
Bayesian gradient estimates from zeroth-order data

- We model $\mathbf{f}$ with a GP prior.
- The GP over $\mathbf{f}$ induces a distribution over $\nabla \mathbf{f}$.
- $\nabla \mathbf{f}$ is again a GP — available in closed form.

$$f \sim \mathcal{GP}(m, k)$$

$$m : \mathcal{X} \rightarrow \mathbb{R}$$

$$k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$$



Reducing gradient uncertainty: the GI acquisition

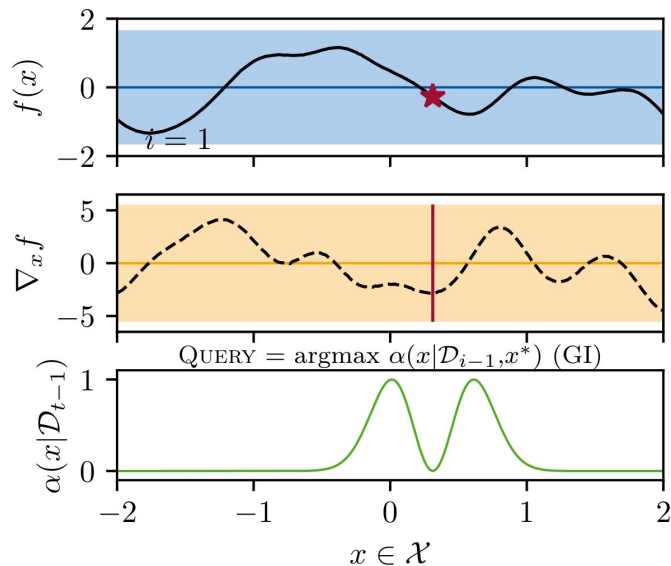
GP-UCB and EI ask: **where is the optimum?** GI asks: **where is the slope?**

- A GIBO query optimally reduces the uncertainty of the **gradient** at the current $\mathbf{x}^*$ (★).

$$\alpha(x \mid \mathcal{D}_{i-1}, x^*) = -\text{Tr} \left(\mathbb{E} \left[\Sigma'(\hat{x}^* \mid \mathcal{D}_i) \right] \right)$$

$$\nabla_x f|_{x=\hat{x}^*} \sim \mathcal{N} \left(m'(\hat{x}^*), \Sigma'(\hat{x}^*) \right)$$

$\Sigma'(\mathbf{x}^*)$ depends only on **where** we query, not on what we observe — so the next query can be planned in advance.

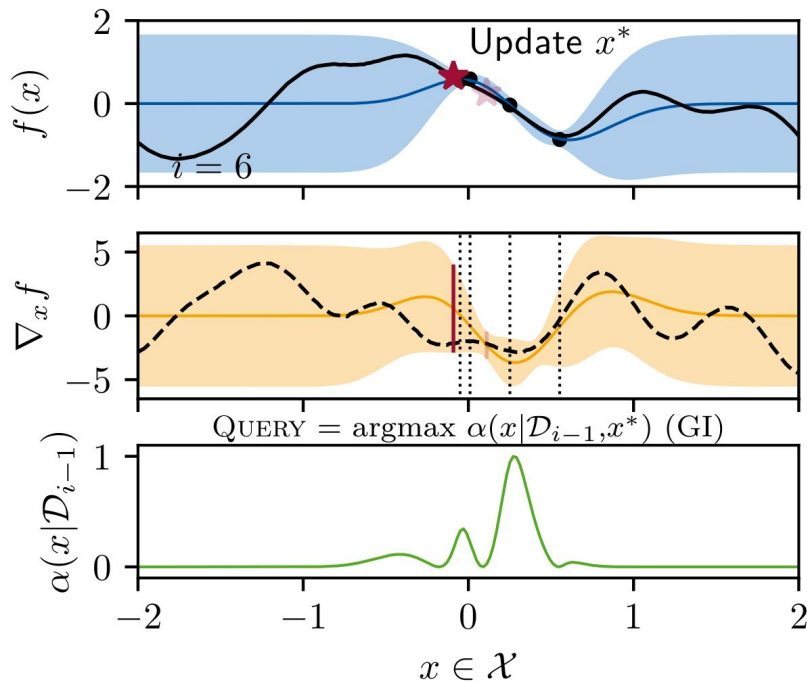


GIBO in action

The GIBO algorithm

Input: initial $\mathcal{D}$, initial x^* (★)

```
1: repeat
2:   for  $t = 1, 2, \dots, T$  do
3:      $x \leftarrow \text{QUERY}(\mathcal{D}, x^*)$ 
4:      $\hat{f}_x \leftarrow \text{OBSERVATION}(x)$ 
5:      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(x, \hat{f}_x)\}$ 
6:   end for
7:    $x^* \leftarrow \text{GradStep}(\mathcal{D}, x^*)$  📁
8: until some stopping criterion
9: return DECISION( $\mathcal{D}$ )
```



What local search costs you

- **No global guarantee.** GIBO converges to a **local** optimum — which one depends on where you start.
- **In exchange,** the query budget scales with the local geometry, not with the volume of the search space.
- **There is theory.** Wu, Kim, Garnett & Gardner (NeurIPS 2023) give convergence rates for GIBO, and show that local optima of GP sample paths are unusually good.
- **Knobs that matter:** inner queries per gradient step, the length scale of the local model, and the step size.

The GIBO family: where to read further

Estimate the gradient, then step — the GIBO family

2021 GIBO — *Local policy search with Bayesian optimization*, Müller, von Rohr & Trimpe — NeurIPS

Query to shrink the gradient posterior, then follow its mean. This talk.

2022 MPD — *Local Bayesian optimization via maximizing probability of descent*, Nguyen, Wu, Gardner & Garnett — NeurIPS

Step along the direction most likely to be a descent direction.

2023 Analysis — *The Behavior and Convergence of Local Bayesian Optimization*, Wu, Kim, Garnett & Gardner — NeurIPS

Convergence rates for GIBO, and why its local optima are good.

2024 MinUCB — *Minimizing UCB: a Better Local Search Strategy in Local Bayesian Optimization*, Fan, Wang, Ng & Hu — NeurIPS

Swap the gradient step for one minimising the UCB; LA-MinUCB adds look-ahead.

Local, but a different idea

2019 TuRBO — *Scalable Global Optimization via Local Bayesian Optimization*, Eriksson, Pearce, Gardner, Turner & Poloczek — NeurIPS

Localizes by shrinking the domain — trust regions, no gradient model.

2025 LES — *Local Entropy Search over Descent Sequences for Bayesian Optimization*, Stenger, Lindicke, von Rohr & Trimpe — ICLR

Entropy search over whole descent sequences. Our recent work.

Code: github.com/avrohr/gibo — GIBO and the baselines from the NeurIPS 2021 paper.

Conclusions

- Bayesian Optimization is a **beautiful structured bandit problem**
- That has **many application** use cases
- The overall idea is simple: model the unknown function and use **uncertainty estimation** to search and optimize
- Lots of interesting **trade-offs** to keep in mind: computational cost of optimization, required target precision,...
- Lots of **practical considerations and modelling choices**

BUT

- [We still don't understand high-dimensional Bayesian Optimization.](#) AISTATS 2026 Best Paper Award

Practical considerations: draft for the practical session

Head to the Colab:

https://colab.research.google.com/drive/14Rw7Gwoguz2xzUSMnWw_hX3HzvR4mYHo_?usp=sharing

